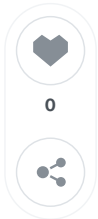


5. 합성곱 신경망 (1)

goblurry · 어제

통계 수정 삭제

CNN 딥러닝



딥러닝 파이토치 교과서

▼ 목록 보기

2/2



🇰🇷 딥러닝 파이토치 교과서 5장 '합성곱 신경망 (1)'의 5.1, 5.2절의 내용을 정리한 포스트입니다. 5장은 두 개의 포스트로 나누어 정리합니다.

- 5.1 합성곱 신경망
- 5.2 합성곱 신경망 맛보기
- 5.3 전이 학습
- 5.4 설명 가능한 CNN
- 5.5 그래프 합성곱 네트워크

4장에서 딥러닝 알고리즘의 일종으로 합성곱 신경망에 대한 간단한 개념을 설명했다.

(<https://velog.io/@hyunsun/4#합성곱-신경망-cnn>)

다시 한번 말하자면, 합성곱 신경망(이하 CNN)은 합성곱층과 풀링층(Pooling layer)을 포함하는 고성능 이미지 처리 인공 신경망 알고리즘이다. 영상과 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체의 위치를 찾아내는 데 유용하다.

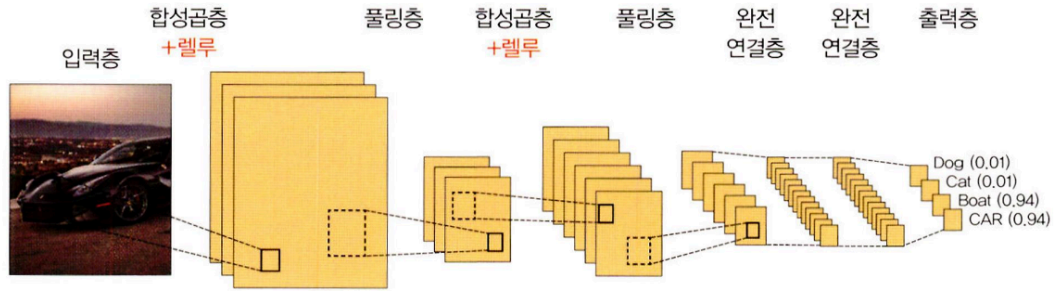
5.1 합성곱 신경망 Convolutional Neural Network

딥러닝 학습의 두 단계로 순전파와 역전파가 있었다. 순전파 과정의 결과로 계산된 오차 정보가 신경망의 노드들로 전송되어 역전파에 사용된다. 이때 계산 과정이 복잡할 뿐 아니라 CPU, GPU, 메모리와 같은 자원 소모량이 크다. 이 문제를 해결하는 데 사용되는 것이 CNN이다.

CNN은 이미지의 전체를 한 번에 계산하는 것이 아니라, 이미지의 국소 부위를 계산하기 때문에 시간과 자원을 절약하면서 이미지의 세밀한 부분까지 분석하는 신경망이다.

합성곱 신경망 구조 및 합성곱층의 필요성

▼ 그림 5-2 합성곱 신경망 구조



CNN은 **다차원 데이터 배열**을 처리하도록 구성되어 컬러 이미지와 같은 다차원 배열 처리에 특화되어 있으며, 위 이미지와 같이 다섯 개의 계층으로 구성된다.

1. 입력층
2. 합성곱층
3. 풀링층
4. 완전연결층
5. 출력층

합성곱 신경망의 작동 단계

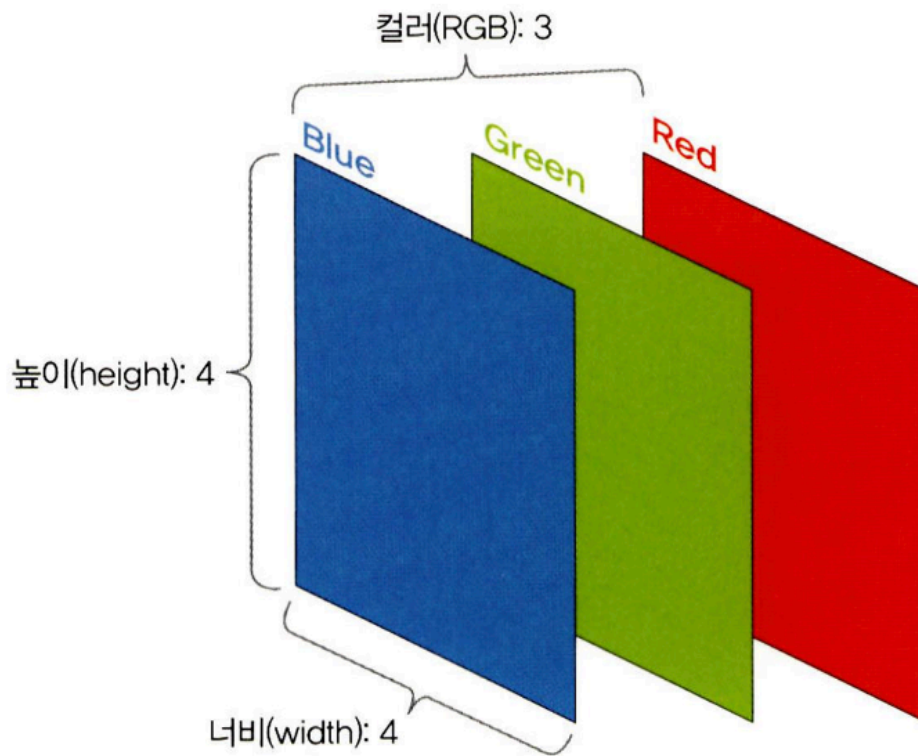
1. **합성곱층과 풀링층**을 거치며 입력 이미지의 주요 특성 벡터(feature vector)를 추출한다.
2. 추출된 주요 특성 벡터들이 **완전연결층**을 거치며 1차원 벡터로 변환된다
3. 활성화 함수인 소프트맥스 함수를 사용해 **출력층**에서 최종 결과가 출력된다.

입력층 Input layer

입력 이미지 데이터가 최초로 거치게 되는 계층으로, 단순 1차원 데이터가 아니라 **높이, 너비, 채널**의 값을 가지는 **3차원 데이터**이다.

이때 채널 값은 이미지가 gray scale일 때 1, rgb(color)일 때 3이다.

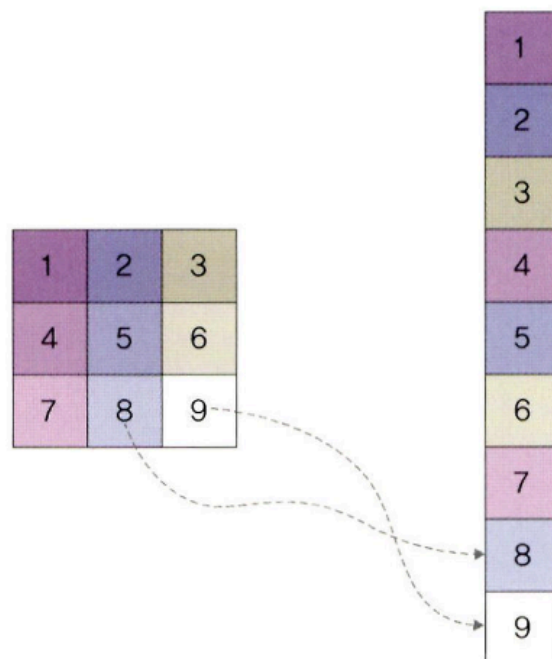
▼ 그림 5-3 채널



이와 같은 이미지 형태는 높이 4, 너비 4, 채널은 rgb이므로 shape은 (4, 4, 3)으로 표현된다.

합성곱층 Convolutional layer

▼ 그림 5-1 합성곱층 원리



합성곱층은 CNN에서 입력층 다음으로 이어지는 계층이다. 이미지 분석은 위 이미지의 좌측 같은 3x3 배열을 우측과 같이 펼쳐서(flattening), 각 픽셀에 가중치를 곱한 뒤 은닉층에 값을 전달한다. 하지만 이렇게 이미지를 펼쳐서 분석하면 **데이터의 공간적 구조를 무시하게 되는 문제점이 발생한다.**



이런 식으로 말이다. (사진 출처:

<https://yeong-jin-data-blog.tistory.com/entry/딥러닝-합성곱-신경망CNN-개념>)

이러한 문제를 방지하기 위해 도입된 것이 합성곱층이다.

합성곱층은 입력층으로 들어온 이미지에서 특성을 추출하는 역할을 한다. 이때 사용하는 것이 **커널**이나 **필터**이다. 커널이나 필터는 이미지의 모든 영역을 훑으면서 특성을 추출하는데, 그 결과물은 **특성 맵 feature map**이라고 한다.

커널은 3x3, 5x5 크기로 적용되는 것이 일반적이다. 필터를 적용하는 위치의 간격을 의미하는 '스트라이드'에 따라 순차적으로 이동한다. 입력 이미지에 필터를 지정된 스트라이드만큼 이동시키면서, 모든 입력 값과 필터 값의 합성곱 연산으로 새로운 특성 맵을 만들게 된다.

다음은 크기가 (6, 6, 1)인 이미지에 3x3 크기의 커널을 스트라이드 1로 적용해 합성곱 연산을 수행하는 과정이다.

1단계: 입력 이미지에 3x3 필터 적용



입력 이미지와 필터를 포개고 대응되는 수끼리 곱한 후 모두 합한다. 합한 값을 특성 맵의 칸에 적는다.

$$(1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) + (0 \times 0) + (0 \times 1) + (0 \times 0) + (1 \times 1) = 3$$

2단계: 필터가 1(스트라이드)만큼 이동

▼ 그림 5-5 입력 이미지에 필터가 1만큼 이동



지정된 스트라이드인 1만큼 우측으로 이동해 같은 연산을 수행한다.

3단계: 필터가 1만큼 또 이동

▼ 그림 5-6 입력 이미지에 필터가 1만큼 두 번째 이동



4단계: 필터가 마지막으로 이동

▼ 그림 5-9 입력 이미지에 필터가 1만큼 마지막으로 이동

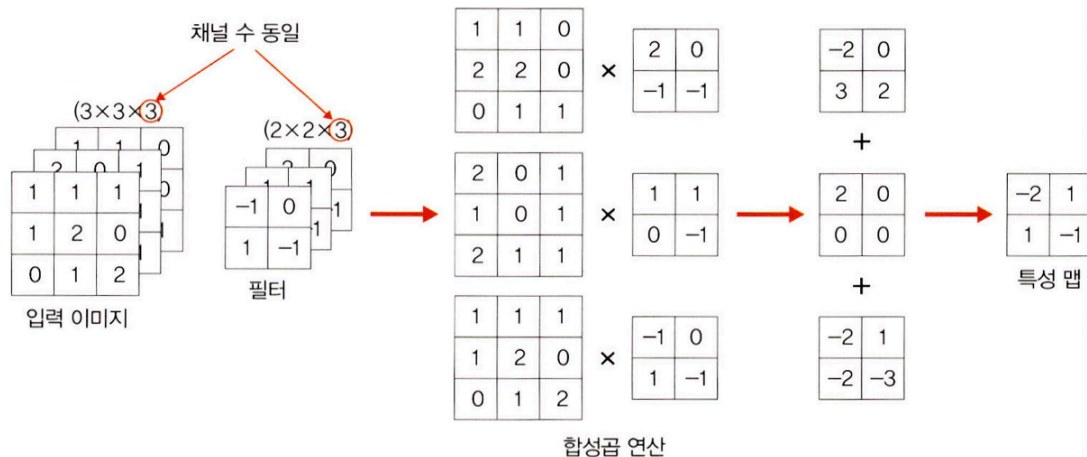


위의 과정을 거쳐, 필터와 스트라이드의 상호작용으로 크기가 (6, 6, 1)였던 원본 이미지 데이터가 (4, 4, 1)의 특성 맵으로 줄어들었다.

앞에서 설명했듯, 채널값이 1인 것은 이미지가 그레이스케일이라는 것이다. 그렇다면 채널 값이 3인, 즉, **컬러 이미지의 합성곱**은 어떻게 이루어질까? 기본적으로 스트라이드와 연산 방법은 동일하다. 하지만 그레이스케일과 구분되는 주요 차별점이 두 가지 있다.

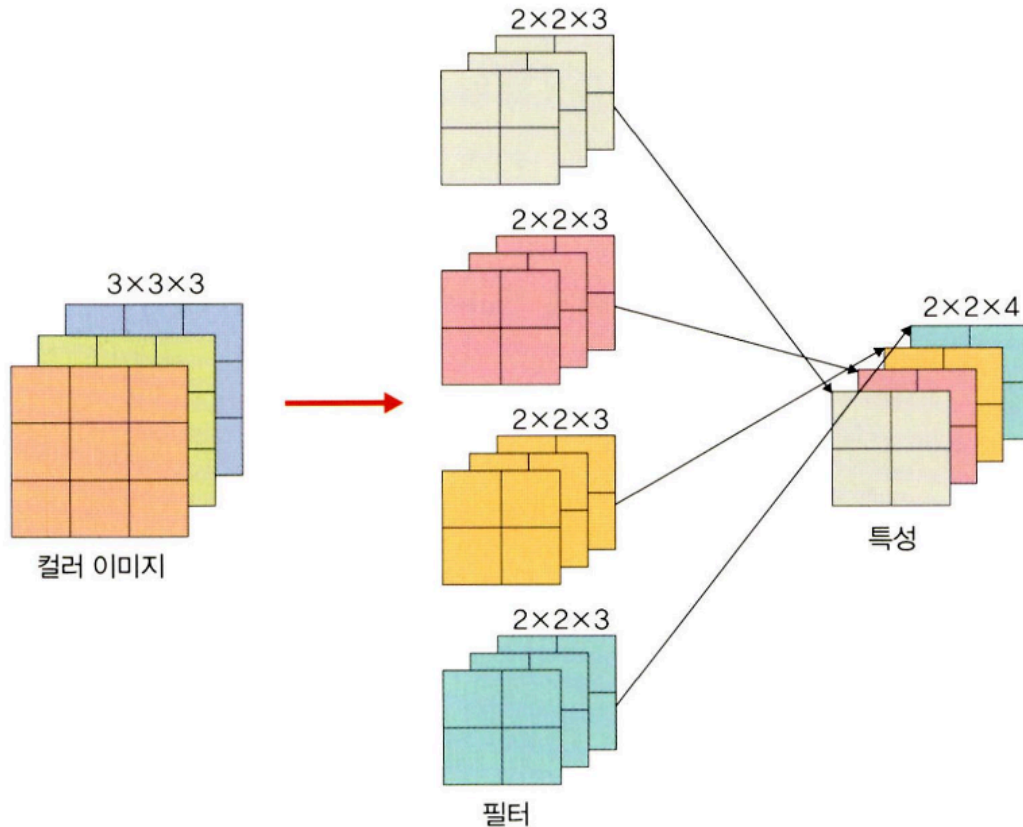
- 필터 채널이 3이다.
- RGB 각각에 적용되는 가중치가 다르다.

▼ 그림 5-10 컬러 이미지 합성곱



입력 채널과 필터의 depth는 같아야 한다. 이때 필터의 채널이 3이라는 것이 필터의 개수가 3개임을 뜻하는 것이 아니다... (rgb면 3, grayscale이면 1일 뿐)

▼ 그림 5-11 필터가 2 이상인 합성곱



필터의 개수가 2개 이상인 합성곱 형태는 다음과 같다. shape이 (2, 2, 3)인 필터가 4개 있고, 이것들이 합성곱 연산에 사용되어 특성 맵도 4개가 출력되었다.

- 입력 데이터의 채널 수와 필터의 채널 수는 같아야 한다.
- 필터 개수는 출력 채널의 개수와 같다.

+) 보충: 필터의 개수

이후에 나올지는 모르겠지만 교재의 해당 부분에서 필터의 개수에 대해서는 자세히 언급하지 않았는데, 필터의 개수가 많아지면 무엇이 달라지는 것인지 궁금해서 관련 내용을 찾아보았다.

필터의 개수가 늘어날수록 이미지의 추상적이고 고차원적인 특징을 더 잘 추출할 수 있다고 한다. 필터가 1개일 때는 딱 한 가지의 특징만 추출하는데, 필터가 여러 개가 되면 각각의 필터에서 다른 특징을 추출한다. 예를 들어 개수가 4개라면 각각 가로 엮지, 세로 엮지, 텍스처, 색상 이런 식으로 말이다.

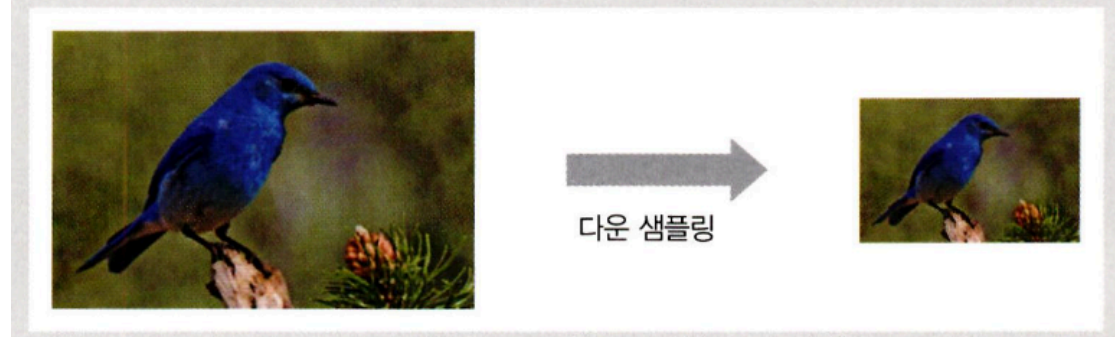
합성곱층에 대한 내용을 정리하자면 다음과 같다.

- 입력 데이터: $W_1 \times H_1 \times D_1$ (가로, 세로, 채널 깊이)
- 하이퍼 파라미터: 필터 개수 K , 필터 크기 F , 스트라이드 S , 패딩 P
- 출력 데이터
 - $W_2 = (W_1 - F + 2P) / S + 1$
 - $H_2 = (H_1 - F + 2P) / S + 1$
 - $D_2 = K$

풀링층 Pooling layer

풀링층은 합성곱층과 유사하게, 특성 맵의 차원을 다운 샘플링(이미지 축소)하여 연산량을 줄이고, 주요 특성 벡터를 추출해 학습 효과를 높인다.

▼ 그림 5-12 다운 샘플링



풀링 연산에는 **최대 풀링**, **평균 풀링** 두 가지가 사용된다.

1. 최대 풀링: 대상 영역에서 최댓값을 추출
2. 평균 풀링: 대상 영역에서 평균을 반환

대부분의 CNN에서는 최대 풀링이 사용된다. 평균 풀링은 각 커널의 값을 평균화시켜 중요 가중치를 가지는 값의 특성이 희미해질 수 있기 때문이다.

다음은 **최대 풀링**의 연산 과정이다.

최대 풀링 1단계:

▼ 그림 5-13 첫 번째 최대 풀링 과정



선택된 영역의 최댓값인 3을 선택한다.

최대 풀링 2단계:

▼ 그림 5-14 두 번째 최대 풀링 과정



최댓값인 12를 선택한다.

최대 풀링 3-4단계:

세 번째 최대 풀링 과정

2, -3, 3, -2 값 중에서 최댓값(3)을 선택합니다.

▼ 그림 5-15 세 번째 최대 풀링 과정



네 번째 최대 풀링 과정

0, 1, 4, -1 값 중에서 최댓값(4)을 선택합니다.

▼ 그림 5-16 네 번째 최대 풀링 과정



평균 풀링을 사용할 경우, 각 영역의 평균값을 사용하기 때문에 0, 3, 0, 1이 출력된다.

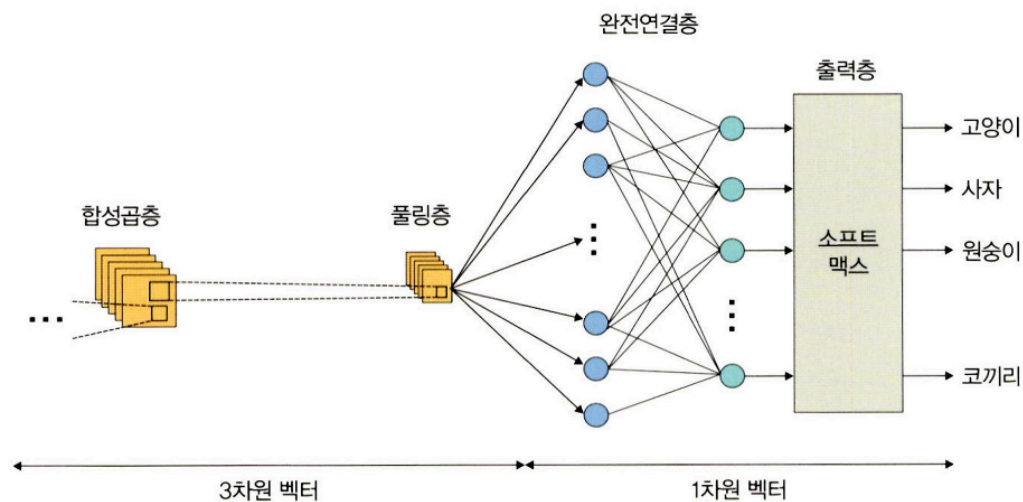
합성곱층에 대한 내용을 정리하자면 다음과 같다. 최대 풀링과 평균 풀링은 계산 과정은 다르지만 사용 파라미터는 동일하다.

- 입력 데이터: $W_1 \times H_1 \times D_1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W_2 = (W_1 - F) / S + 1$
 - $H_2 = (H_1 - F) / S + 1$
 - $D_2 = D_1$

완전연결층 Fully connected layer

합성곱층과 풀링층을 거치며 차원이 축소된 특성 맵은, 최종적으로 완전연결층으로 연결된다. 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게 된다.

♥ 그림 5-18 완전연결층



출력층 Output layer

마지막 계층인 출력층에서는 소프트맥스 활성화 함수가 사용된다.

*소프트맥스 함수: 입력값을 0-1 사이에 출력되도록 정규화하여 출력값들의 총합이 항상 1이 되도록 하는 함수. 이미지가 각 레이블(label)에 속할 확률 값이 출력되고, 그중 최댓값을 가지는 레이블이 최종 값으로 출력된다.

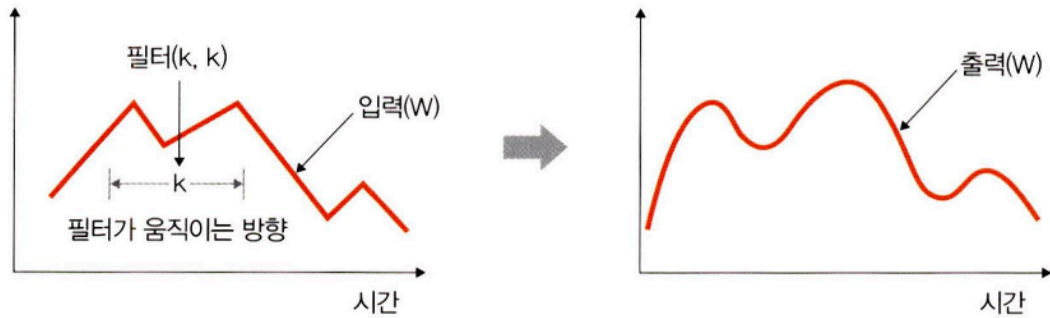
1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류할 수 있다.

1. 1D 합성곱

필터가 시간 축으로 좌우로만 이동할 수 있는 합성곱이다. 입력 W 와 필터 k 에 대한 출력은 W 이다. 출력은 1차원의 배열이 되고, 그래프의 곡선을 완화하는 경우에 많이 사용된다.

▼ 그림 5-19 1D 합성곱

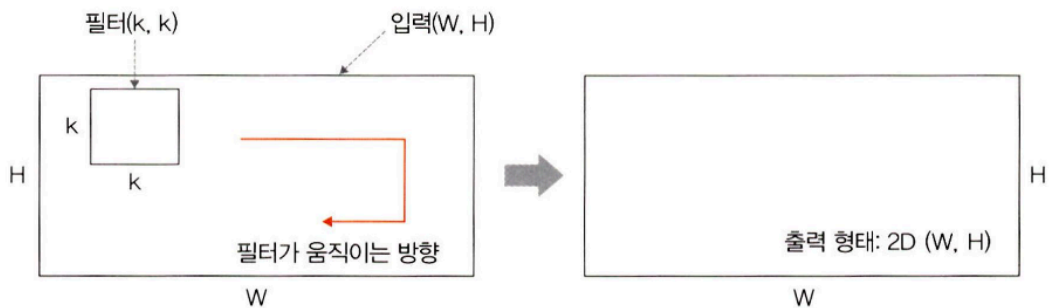


- 입력: W 너비(Width)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width)

2. 2D 합성곱

필터가 방향 2개로 움직이는 형태이다. 입력 (W, H) 와 필터 (k, k) 에 대한 출력은 (W, H) 가 되고, 2차원 행렬 형태로 값이 출력된다.

▼ 그림 5-20 2D 합성곱

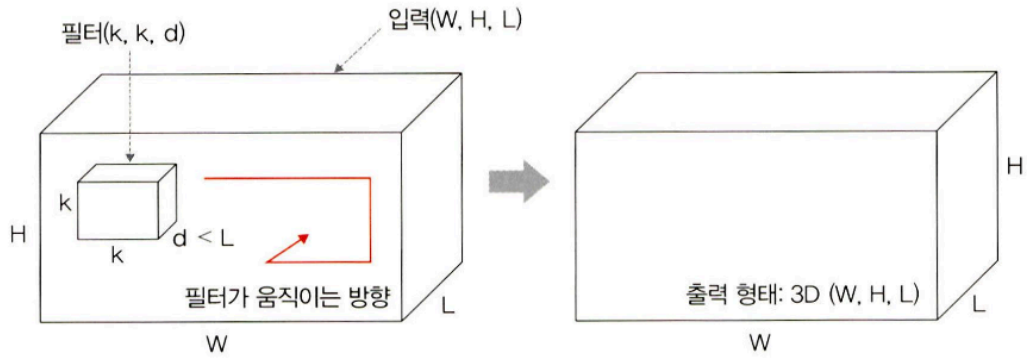


- 입력: W 너비(Width), H 높이(Height)
- 필터: $k \times k$ (높이 $\times$ 너비)
- 출력: W 너비(Width), H 높이(Height)

3. 3D 합성곱

필터가 움직이는 방향이 세 개 있다. 입력 (W, H, L) 에 필터 (k, k, d) 를 적용하면 출력으로 (W, H, L) 형태가 나온다. (3차원 배열) 이때 $d < L$ 을 유지하는 게 중요하다.

♥ 그림 5-21 3D 합성곱

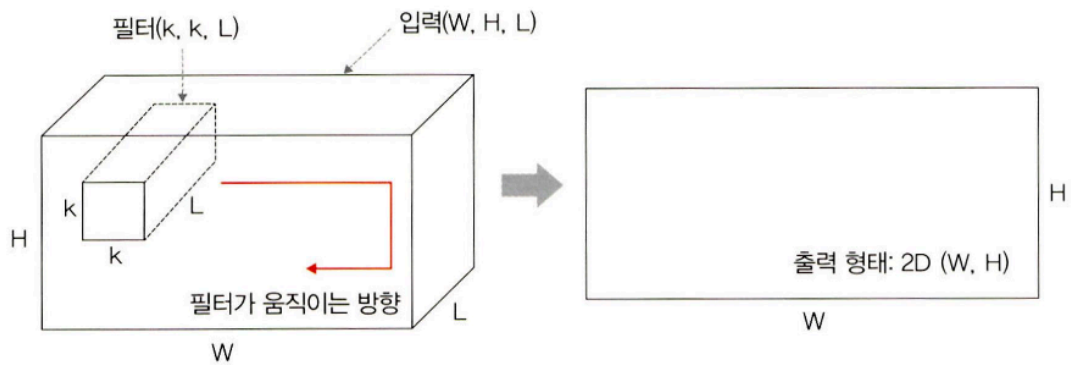


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), d: 깊이(depth)
- 출력: W 너비(Width), H 높이(Height), L 길이(Length)

4. 3D 입력을 갖는 2D 합성곱

입력 (W, H, L)에 필터 (k, k, L)를 적용했을 때 2차원 행렬 (W, H)가 출력되는 합성곱의 형태가 있다. 필터는 두 방향으로 움직인다. 대표적 사례로 LeNet-5와 VGG가 있는데 6장에서 다룬다고 한다.

♥ 그림 5-22 3D 입력을 갖는 2D 합성곱

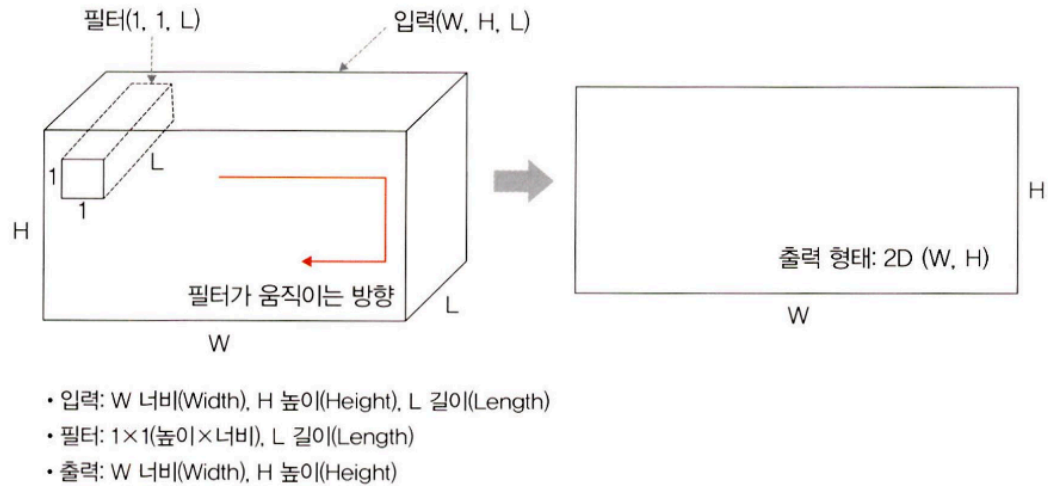


- 입력: W 너비(Width), H 높이(Height), L 길이(Length)
- 필터: $k \times k$ (높이 $\times$ 너비), L 길이(Length)
- 출력: W 너비(Width), H 높이(Height)

5. 1x1 합성곱

3차원 형태의 입력 (W, H, L)에 필터 (1, 1, L)를 적용하여 (W, H)의 출력을 얻는 방식이다. 채널 수를 조절함으로써 연산량이 감소되는 효과가 있으며, 대표적 사례로 GoogLeNet이 있다. 마찬가지로 6장에서 다룬다고 한다.

♥ 그림 5-23 1×1 합성곱



5.2 합성곱 신경망 맛보기

2절에서는 파이토치에 내장된 `fashion_mnist` 데이터셋을 이용해 합성곱 신경망을 직접 구현해 본다. 이 데이터셋은 운동화, 셔츠, 샌들 같은 작은 이미지들의 모음으로, 10개의 라벨로 분류될 수 있는 28x28픽셀의 이미지 7만 개로 구성되어 있다.

훈련 데이터는 0-255 사이 값을 가지는 넘파이 배열이며, 정답 데이터는 0-9 사이의 정수 값을 가지는 배열이다.(이미지의 클래스를 나타냄)

교재에서는 데이터셋을 로드하는 단계부터 시작하여 분류에 사용할 클래스 정의, 심층 신경망 구현(합성곱 신경망과의 성능 비교를 위해)까지 진행하나 본 포스팅에서는 합성곱 신경망 구현에 대해서만 설명하려고 한다. 전체 소스 코드는 https://github.com/goblurry/DL_study 에서 볼 수 있다.

각 이미지에 대한 레이블은 다음과 같이 열 개의 클래스로 구성된다.

```
labels_map = {0: 'T-Shirt', 1: 'Trouser', 2: 'Pullover', 3: 'Dress', 4: 'Coat',
              5: 'Sandal', 6: 'Shirt', 7: 'Sneaker', 8: 'Bag', 9: 'Ankle Boot'}
```

심층 신경망 구현하기

```
# 합성곱 네트워크 생성
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1), # Conv2d 적용
            nn.BatchNorm2d(32), # 평균과 분산을 이용해 정규화
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2) # 이미지 크기 축소
        )

        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
```

```

        nn.BatchNorm2d(64),
        nn.ReLU(),
        nn.MaxPool2d(2)
    )

    self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
    self.drop = nn.Dropout2d(0.25)
    self.fc2 = nn.Linear(in_features=600, out_features=120)
    self.fc3 = nn.Linear(in_features=120, out_features=10) # 10: 클래스 개수

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

```

이 코드를 보면 층 2개로 나누어져 있다. (layer1, layer2) layer 1에서는 원본 이미지를 입력으로 받아서 32개의 피쳐로 반환한다. layer 2에서는 그 32개의 특징을 입력으로 바탕으로 64개의 더 복잡한 특징을 추출한다.

nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1) 이 합성곱 연산을 수행하는 함수로, 이미지의 특징을 추출한다. 보다시피 in_channels 값이 1이다. 이 데이터는 흑백 이미지로 이루어져 있기 때문이다. (rgb일 경우 3)

kernel_size 파라미터는 커널(필터)의 크기를 의미한다. 커널은 이미지 특징을 찾아내기 위한 공용 파라미터이며, CNN에서 학습 대상이 된다.

💡 이 라인의 경우 (3×3×1) 크기의 커널 32개로 구성된 합성곱 계층으로, 흑백 이미지를 입력으로 받아, 32개의 서로 다른 특징을 추출해서 채널 32짜리 출력을 생성하는 것이다.

nn.MaxPool2d(kernel_size=2, stride=2) 는 이미지 크기를 축소시키는(다운샘플링) 용도로 사용한다. 이 라인에서는 이미지를 2×2 영역씩 잘라서 그 안에서 최댓값을 찾는다.

self.fc로 시작하는 라인이 3개 있는데, 이 fc가 fully connected, 즉 완전연결층이다. 모든 입력을 다 연결해서 최종 판단을 내리는 부분이다.

conv에서 특징을 추출하고, pool에서 압축하고, fc에서 그 정보를 이용해 정답을 결정한다.

```

# 합성곱 네트워크를 위한 파라미터 정의
learning_rate = 0.001;
model = FashionCNN();
model.to(device)

criterion = nn.CrossEntropyLoss();
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate);
print(model)

```

코드를 입력하세요

정의한 CNN 구조를 model에 로드하고, 예측값과 정답 간의 손실 계산을 위해 criterion을 설정한다. 마지막으로 가중치를 업데이트하는 알고리즘(Adam)을 optimizer에 정의한다. 출력 결과는 다음과 같다.


```

... FashionCNN(
    (layer1): Sequential(
      (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
      (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU()
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (layer2): Sequential(
      (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU()
      (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    )
    (fc1): Linear(in_features=2304, out_features=600, bias=True)
    (drop): Dropout2d(p=0.25, inplace=False)
    (fc2): Linear(in_features=600, out_features=120, bias=True)
    (fc3): Linear(in_features=120, out_features=10, bias=True)
  )

# CNN에 데이터 적용해 모델 학습시키기
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        train = Variable(images.view(100, 1, 28, 28)) # 이미지를 CNN 입력 형태로 변환
        labels = Variable(labels)

        # 학습 데이터를 모델에 적용
        outputs = model(train) # 순전파
        loss = criterion(outputs, labels)
        optimizer.zero_grad() # 역전파
        loss.backward()
        optimizer.step()
        count += 1

    # 성능 평가
    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device) # 확률 최대인 클래스 선택
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}%".format(count, loss.data, accuracy))

```

CNN 모델에 데이터를 적용해 학습시키는 부분이다.

입력 데이터를 모델에 전달해 예측값을 생성하고, 손실 함수를 통해 오차를 계산한 뒤 역전파를 통해 가중치를

업데이트한다. 이 과정을 여러 epoch 동안 반복하며 모델을 점진적으로 최적화한다. 또한 일정 반복마다 테스트 데이터를 이용해 정확도를 측정함으로써 모델의 성능을 평가한다.

```
... Iteration: 500, Loss: 0.2391732633113861, Accuracy: 91.05000305175781%
    Iteration: 1000, Loss: 0.18842992186546326, Accuracy: 89.87999725341797%
    Iteration: 1500, Loss: 0.06646131724119186, Accuracy: 90.02999877929688%
    Iteration: 2000, Loss: 0.1638677567243576, Accuracy: 90.27999877929688%
    Iteration: 2500, Loss: 0.1492127627134323, Accuracy: 89.77999877929688%
    Iteration: 3000, Loss: 0.11005957424640656, Accuracy: 89.80000305175781%
```

정확도가 약 89%로 출력된다. 이 포스팅에서는 생략했지만, 심층 신경망 네트워크의 경우 정확도가 86%였다. 차이가 많이 나지 않기 때문에 간편한 심층 신경망을 쓰면 되지 않나 싶을 수 있지만, 이미지 데이터가 많아지면 심층 신경망으로는 정확한 특성 추출과 분류가 불가능해 합성곱 신경망을 사용하는 것이 좋다고 한다.



goblurry



이전 포스트

4. 딥러닝 시작

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate